Koneru Lakshmaiah Education Foundation
(Category -1, Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
❖ Approved by AICTE ❖ ISO 21001:2018 Certified
Campus: Green Fields, Vaddeswaram - 522 302, Guntur District, Andhra Pradesh, INDIA.
Phone No. +91 8645 - 350 200; www.klef.ac.in; www.klef.edu.in; www.kluniversity.in
Admin Off: 29-36-38, Museum Road, Governorpet, Vijayawada - 520 002. Ph: +91 - 866 - 3500122, 2576129

SMART IRRIGATION SYSTEM

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

By

SOMISETTI VENKAT ANISH 2601150089

under the supervision of

Dr RAJU ANITHA

Professor

Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation
(KLEF) Vaddeswaram, Guntur District,
Andhra Pradesh, India.

DECLARATION

I hereby declare that the project report entitled “Smart Irrigation System” is prepared for academic purposes under the guidance of Raju Anitha. This report documents the proposed system architecture, control methodology, simulation setup, reported simulation results, and planned field-evaluation procedure.

I further declare that the material used in preparing this report has been acknowledged through references wherever applicable. The numerical performance values presented in the results chapter are explicitly identified as outputs from a simplified simulation using assumed parameters; they are not presented as field measurements. Any future hardware validation will be documented separately using recorded experimental data.

Place: Vaddeswaram

Date: 14-10-2026

Signature of the Student: Somisetti Venkat Anish 2601150089

INDEX

S. No.	Topics	Page No.
1	Introduction and Problem Definition	1 - 3
2	Literature Review and Theoretical Foundations	4 - 5
3	Proposed Methodology and System Architecture	6 – 10
4	Experiment setup	11 - 14
5	Results	15 - 17
6	Conclusion and Future Work	18
7	References	19

LIST OF TABLES

S. No.	Table Name	Section
1	Hardware components and their roles	3.2
2	Components	3.4
3	Condition Values	3.5
4	Tools and Technologies	3.7
5	Simulation parameters	4.2
6	Simulation results	4.4
7	Planned field evaluation metrics	4.5

LIST OF FIGURES

Figure 3.1	Logical architecture of the IoT-based smart irrigation system.
Figure 3.2	Typical hardware connection diagram and configurable ESP32 pin mapping.
Figure 3.3	Control flow of the irrigation controller, including safety checks.
Figure 4.1	Simulated soil moisture under threshold control and a fixed schedule.
Figure 5.1	Soil Moisture Levels
Figure 5.2	Five-Sensor Simulation
Figure 5.3	Water Usage Comparison
Figure 5.4	Sensor Polling Performance

ABSTRACT

Fixed-schedule irrigation may apply water when the soil is already sufficiently moist or fail to respond promptly during dry conditions. This project presents a low-cost Internet of Things (IoT)-based smart irrigation system that uses an ESP32 microcontroller, a capacitive soil-moisture sensor, an air temperature and humidity sensor, a water-level sensor, and a relay-controlled water pump. The controller uses an on-off hysteresis strategy with separate lower and upper moisture thresholds to avoid frequent switching. Safety interlocks stop the pump when sensor readings are invalid, the tank level is low, or the maximum continuous pump runtime is exceeded.

The system publishes moisture readings, environmental measurements, pump state, tank status, and alerts through MQTT. A dashboard can display telemetry and provide manual overrides or threshold updates, while local control continues even if Wi-Fi connectivity is temporarily unavailable. The control logic is evaluated using a discrete-time soil-water simulation over three days. Under the stated assumed parameters, the threshold controller used 120 pump-on minutes compared with 260 minutes for a fixed twice-daily schedule, and the simulated time within the 35–60% target moisture band was 98.8% compared with 97.2%. These figures are simulation outputs, not field measurements.

The report describes the problem, related work, theoretical basis, architecture, control algorithm, simulation assumptions, results, limitations, and a protocol for future field testing. The main contribution is a reproducible low-cost baseline that combines local control, safety interlocks, and IoT telemetry while clearly separating modeled results from evidence that must be collected from a physical prototype.

Keywords: Smart irrigation, Internet of Things, ESP32, soil moisture sensor, MQTT, hysteresis control, precision agriculture, water conservation.

1. INTRODUCTION AND PROBLEM DEFINITION

1.1 Background

Water is a critical resource in agriculture, gardening, and small-scale cultivation. Irrigation supports plant growth when rainfall is insufficient, but applying water without considering the current soil condition can lead to unnecessary consumption, energy use, and drainage. In many small installations, irrigation is performed manually or by a fixed timer. Such methods are simple to operate but do not respond directly to changing soil moisture, rainfall, or environmental conditions.

The Internet of Things enables physical devices to measure conditions, process readings, and exchange information through a network. A low-cost microcontroller with wireless connectivity can collect soil moisture and environmental data and use these readings to make irrigation decisions. In this project, an ESP32 acts as the edge controller. It reads the sensors, evaluates safety conditions, controls a relay-driven pump, and sends telemetry to an MQTT broker for dashboard presentation.

The project focuses on an understandable and reproducible baseline rather than a complex predictive model. A two-threshold hysteresis controller is selected because it is computationally light, easy to configure, and suitable for a microcontroller. A lower threshold starts irrigation when the soil becomes too dry, while an upper threshold stops irrigation after the desired moisture level is reached.

Soil texture, root depth, plant growth stage, sunlight, wind, and recent rainfall can change how quickly water is lost or retained. A timer-based system treats these changing conditions as if they were identical, whereas a sensor-driven controller uses a measurement of the current root-zone condition as feedback. This does not eliminate the need for agronomic knowledge, but it provides a more responsive basis for deciding when to operate the pump. The proposed prototype is intended for a small garden, nursery, or single experimental crop bed. The ESP32 is suitable for this role because it combines digital processing and Wi-Fi connectivity in a compact development board. The local controller can make a decision quickly without sending every measurement to a remote server first. Network communication is used mainly for observation, logging, alerts, and configuration, which helps separate time-critical control from optional dashboard services.

1.2 Problem Definition

A fixed irrigation schedule does not measure the soil before operating the pump. It may therefore irrigate immediately after rainfall or continue to use the same duration despite changes in temperature, soil type, and crop water requirements. Manual monitoring can also be inconsistent and may not provide a timely response when the soil becomes dry.

The system must measure soil moisture, decide whether irrigation is needed, operate the pump safely, and provide status information to the user. It must also prevent unsafe operation when sensor values are invalid, the water tank is empty, or the pump runs for too long. Remote monitoring should be useful without making the basic irrigation decision dependent on network availability.

The engineering problem therefore has both a control requirement and a reliability requirement. A moisture reading may be noisy, a sensor wire may become disconnected, and a water source may be depleted while the controller is running. If such conditions are not handled, an automated system could waste water, damage a pump, or leave the crop without irrigation. The design must prioritize safe shutdown whenever the input data cannot be trusted or a required resource is unavailable.

1.3 Objectives

- Design a low-cost irrigation controller around an ESP32 and a capacitive soil-moisture sensor.
- Calibrate sensor readings and convert them into a relative moisture percentage.
- Implement hysteresis control with configurable lower and upper thresholds.
- Add safety interlocks for invalid readings, low tank level, and maximum pump runtime.
- Publish telemetry and alerts using MQTT and support a dashboard for monitoring and remote overrides.
- Evaluate the control logic using a clearly documented simulation before conducting physical field tests.
- Define a repeatable field-evaluation protocol for sensor error, actuation latency, telemetry delivery, water use, and fault handling.

1.4 Scope of the Project

The scope covers one irrigation zone with a sensor-driven pump, local control logic, MQTT communication, and a web or mobile dashboard. The design includes soil moisture, temperature,

humidity, and water-level sensing. Simulation is used to test the decision logic under controlled assumptions. Actual crop-specific water savings, sensor accuracy, and pump flow rate must be established through field measurements.

The present design does not claim to implement weather forecasting or a full crop evapotranspiration model. The controller reacts to measured soil moisture. Multi-zone operation, solar power, flow-meter-based water measurement, secure remote access, and advanced scheduling are identified as future extensions.

1.5 Existing System

A conventional timer-based system activates a pump at predefined times and for predefined durations. It is easy to deploy but does not consider whether the soil already contains sufficient water. Manual irrigation provides flexibility but depends on user availability and repeated observation. Some sensor-based systems measure moisture but may omit fault handling, remote telemetry, or a clear separation between simulation and field evidence.

1.6 Proposed System

The proposed system combines sensor-based local control with MQTT telemetry. At each sampling interval, the ESP32 reads and filters sensor values, checks the safety interlocks, applies the hysteresis rule, updates the relay, and publishes the current status. Remote commands use the same safety checks as automatic operation, so an override cannot bypass an empty-tank or invalid-sensor protection.

1.7 Advantages

- Responds to soil condition instead of relying only on clock time.
- Uses two thresholds to reduce rapid pump switching near a single set point.
- Provides fail-safe pump-off behavior when important faults are detected.
- Supports low-overhead monitoring through MQTT.
- Continues local irrigation decisions during temporary Wi-Fi outages.

LITERATURE REVIEW AND THEORETICAL FOUNDATIONS

2.1 IoT-Based Irrigation Systems

Research on IoT-based irrigation describes architectures that combine sensing nodes, communication networks, processing platforms, and irrigation actuators. Platform-scale systems demonstrate how data can be collected from multiple field locations and exposed through software services. Device-scale prototypes commonly use microcontrollers, soil sensors, relays, and pumps to automate a small irrigation zone. The present project follows the device-scale approach and emphasizes clear local control behavior, fault handling, and a documented path to field evaluation.

2.2 Irrigation Scheduling Approaches

Irrigation can be scheduled using fixed calendars, soil-moisture thresholds, or a crop-water-balance model. Fixed schedules are straightforward but cannot react directly to rainfall or current soil moisture. Soil-moisture threshold control responds to the measured condition and needs relatively few parameters. Crop-water-balance methods can account for weather, crop coefficients, and evapotranspiration, but require additional data and calibration. This project adopts threshold control as a practical baseline and treats evapotranspiration-based scheduling as future work.

2.3 Soil Water and Thresholds

Soil water availability is bounded by agronomic conditions such as field capacity and the permanent wilting point. The controller should maintain moisture within a crop- and soil-appropriate operating range. Since soil properties vary, the lower and upper thresholds must be configurable rather than assumed to be universally correct.

2.4 Sensor Calibration

A capacitive moisture sensor produces an analog signal that changes with soil moisture. The controller averages multiple analog-to-digital converter samples to reduce noise. Let R_j denote the j-th reading and N the number of samples. The average reading is $\bar{R} = (1/N) \Sigma R_j$. Using dry and wet calibration readings, a relative moisture index can be calculated as $m = 100 \times (R_{dry} - \bar{R})/(R_{dry} - R_{wet})$, clipped to the interval 0–100%. This value is a relative index and should not be treated as an absolute volumetric water-content measurement without appropriate calibration.

2.5 Hysteresis Control

A single threshold can cause the pump to switch repeatedly when the sensor reading fluctuates around the set point. Hysteresis introduces two thresholds. If moisture falls below T_{low}, the pump

turns on. If moisture rises above T_{high}, the pump turns off. Between these thresholds, the previous pump state is retained. The difference between the thresholds creates a stable operating band and reduces unnecessary switching.

The control rule is: $u(k) = 1$ when $m(k) < T_{low}$; $u(k) = 0$ when $m(k) > T_{high}$; otherwise $u(k) = u(k-1)$. Here, $u(k)$ is the pump command and $m(k)$ is the measured moisture index at step k.

2.6 Simplified Soil-Water Balance

For simulation, the root-zone moisture is represented by a discrete-time balance: $m(k+1) = m(k) + \rho u(k) + r(k) - e(k) - d(k)$. Here, ρ is the moisture gain while pumping, $r(k)$ represents rainfall gain, $e(k)$ represents evapotranspiration loss, and $d(k)$ represents drainage loss. This model is intentionally simplified. It is useful for exercising the control logic but does not predict the behavior of a specific field unless its parameters are calibrated against measured data.

2.7 MQTT and Edge Control

MQTT is a lightweight publish/subscribe protocol suitable for constrained devices. The controller publishes telemetry to a zone-specific topic and subscribes to command and configuration topics. Local control is retained on the ESP32 so that the pump can still respond to sensor readings when the network is unavailable. A deployment outside a controlled laboratory network should use broker authentication, encrypted transport such as TLS, and authorization rules for remote commands.

2.8 Summary of Related Work

Existing research supports the use of IoT sensing and soil-moisture-driven irrigation, but performance depends on sensor calibration, actuator sizing, communication reliability, and field conditions. The proposed design combines these practical concerns in a small architecture and distinguishes simulation evidence from future field measurements.

PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE

3.1 Methodology Overview

The system follows a sense–decide–act–report loop. The controller reads the sensors, filters the readings, converts the moisture signal, checks safety conditions, applies hysteresis, drives the relay, publishes telemetry, and processes incoming configuration or override commands. Read multiple ADC samples and calculate the average sensor reading.

1. Convert the average reading into a relative moisture percentage.
2. Check whether the sensor reading is valid and whether sufficient water remains in the tank.
3. Apply the lower and upper moisture thresholds to determine the pump state.
4. Stop the pump if the maximum continuous runtime is exceeded.
5. Drive the relay and publish sensor values, pump state, tank status, and any alert.
6. Wait for the next sampling interval and repeat.

Table : 3.2 Hardware Architecture

Component	**Role in the System**
ESP32 DevKit	Edge controller; ADC sampling, control logic, Wi-Fi and MQTT client.
Capacitive soil-moisture sensor	Analog signal related to soil moisture; primary control input.
DHT11 sensor	Air temperature and relative-humidity monitoring.
Water-level sensor	Detects low tank level to prevent pump dry running.
5 V relay module	Switches the pump supply under controller command.
DC water pump	Delivers water through a hose, valve, or drip line.
Wi-Fi router and MQTT broker	Network path for telemetry and commands.
Web/mobile dashboard	Displays readings, status, alerts, and configurable thresholds.

The pump should use a separate power supply appropriate to its electrical requirements. The relay isolates the pump switching path from the microcontroller output. The final wiring must follow the ratings of the selected pump and relay, and mains-powered equipment should be handled using suitable electrical safety practices.

3.3 Logical Architecture

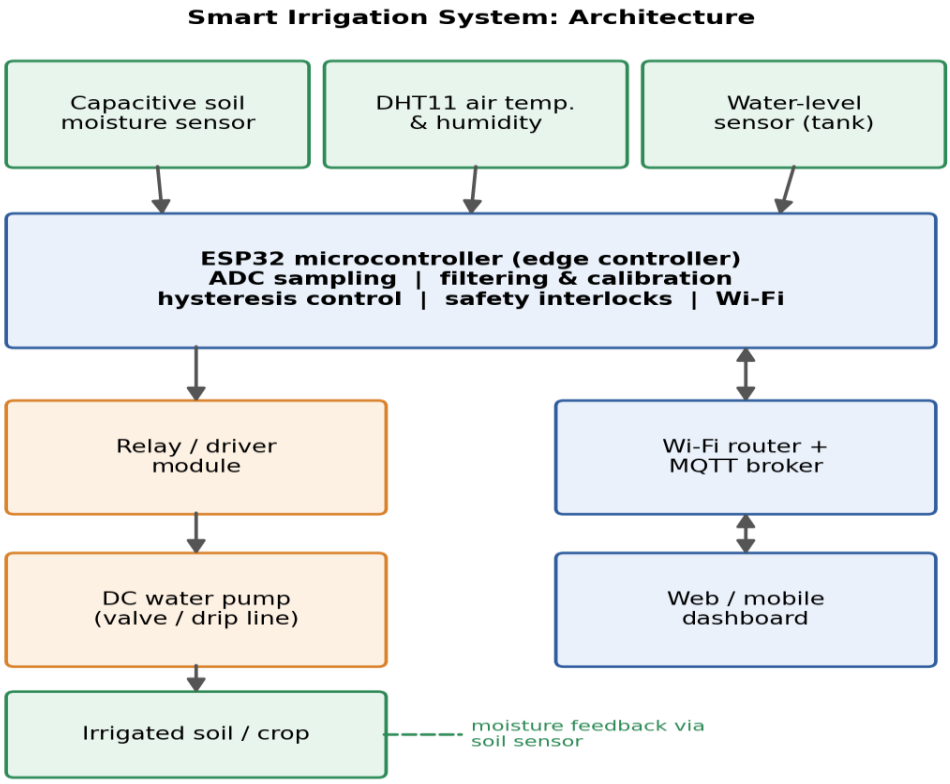


Figure 3.1: Logical architecture of the IoT-based smart irrigation system.

The sensing layer contains the soil-moisture, temperature/humidity, and tank-level sensors. The edge-control layer contains the ESP32 and the firmware. The actuation layer includes the relay and pump. The network and application layer includes Wi-Fi, the MQTT broker, and a dashboard. Sensor readings flow toward the controller; pump commands flow from the controller toward the relay; telemetry flows to the broker and dashboard; and remote configuration messages flow back to the controller.

Sensors → ESP32 Controller → Safety Checks → Hysteresis Decision → Relay → Pump

ESP32 ⇄ Wi-Fi / MQTT Broker ⇄ Dashboard

3.4 Typical Pin Mapping

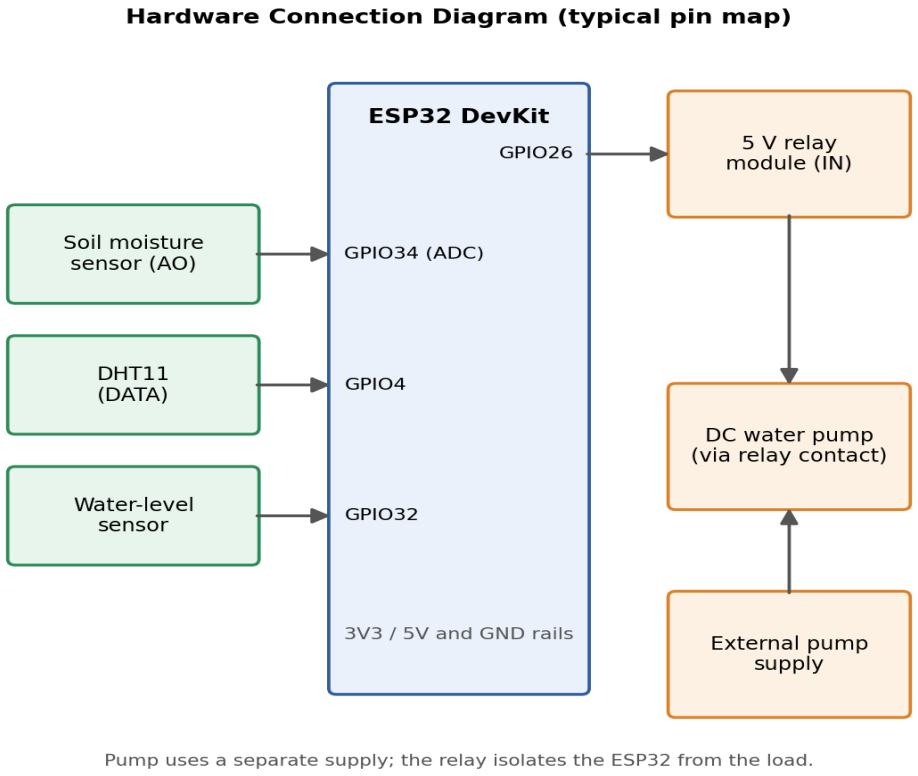


Figure 3.2: Typical hardware connection diagram and configurable ESP32 pin mapping.

Table 3.4 : Components

Device	**Example ESP32 Connection**	**Purpose**
Soil-moisture sensor	GPIO34 / ADC	Analog moisture reading
DHT11 data	GPIO4	Temperature and humidity data
Water-level sensor	GPIO32	Tank-level status input
Relay input	GPIO26	Pump switching control
Power and ground	As required by module	Common reference and suitable supply

Pin assignments are illustrative and configurable. Before hardware operation, verify the selected board pin capabilities, sensor output voltage, relay input requirements, and power-supply ratings.

3.5 Control Algorithm and Safety Interlocks

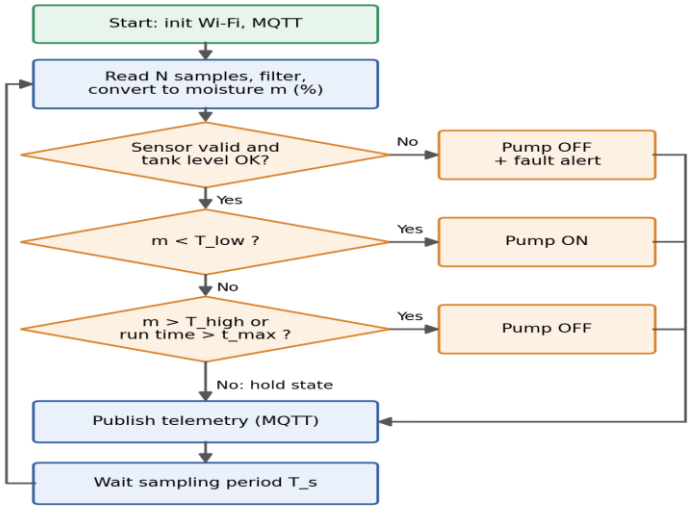


Figure 3.3: Control flow of the irrigation controller, including safety checks.

Three safety checks take precedence over normal threshold control. First, an invalid sensor reading forces the pump off and raises an alert. Second, a low tank level forces the pump off to prevent dry running. Third, a maximum continuous runtime stops the pump even if the moisture target has not been reached. These interlocks reduce the consequences of disconnected sensors, empty tanks, stuck readings, or leaks.

Table 3.5 : Condition of the Values

Condition	**Controller Action**
Sensor reading invalid or outside calibrated range	Pump OFF; publish sensor fault alert.
Tank level below safe threshold	Pump OFF; publish low-water alert.
Moisture below T_low and no fault active	Pump ON; start or continue runtime timer.
Moisture above T_high	Pump OFF.
Moisture between thresholds	Retain previous pump state.
Pump-on duration exceeds t_max	Pump OFF; publish runtime-limit alert.

3.6 Communication and Dashboard

Telemetry should be encoded as a structured JSON message containing moisture, air temperature, humidity, pump state, tank status, timestamp, and active alerts. The controller publishes this information to a per-zone MQTT topic.

Table : 3.7 Tools and Technologies

Technology	Purpose
ESP32	Microcontroller and Wi-Fi connectivity.
Arduino framework	Firmware development and hardware interfaces.
Capacitive soil sensor	Soil-moisture measurement.
DHT11	Air temperature and relative-humidity readings.
MQTT	Lightweight telemetry and command messaging.
Eclipse Mosquitto	MQTT broker suitable for a prototype.
Dashboard application	Visualization and remote threshold configuration.
Python or spreadsheet tool	Simulation, analysis, and plotting of results.

3.8 Pseudocode

- Initialize sensors, Wi-Fi, MQTT, calibration constants, thresholds, and pump state = OFF.
- At each sampling interval, read N sensor samples and compute their average.
- Convert the average reading into relative moisture m.
- If m is invalid or the tank is low, set pump = OFF and publish an alert.
- Otherwise, if m < T_low, set pump = ON and record the start time.
- If m > T_high, set pump = OFF.
- If pump is ON longer than t_max, set pump = OFF and publish a runtime alert.
- Update the relay output and publish telemetry.
- Process authorized remote configuration or override commands through the same safety check.

EXPERIMENTAL SETUP

4.1 Experimental Objective

The initial evaluation checks whether the control logic responds correctly to moisture thresholds and whether its behavior differs from a fixed irrigation schedule under the same modeled soil-water conditions. Since field measurements are not yet available, the reported numerical comparison is based on a discrete-time simulation. The simulation is used to test the controller logic and is not evidence of measured agricultural water savings.

4.2 Simulation Setup

One simulation step represents 10 minutes, and the total horizon is three days. The threshold controller is compared with a fixed schedule that pumps for 40 minutes at 06:00 and 18:00 each day. Evapotranspiration follows a half-sine profile over 06:00–18:00, and a rainfall event occurs on day two from 10:00 to 11:00. All model parameters are assumptions selected to exercise the control logic.

Table 4.2 : Parameter Values

Parameter	**Assumed Value**
Control step / horizon	10 minutes / 3 days
Initial moisture	50%
Lower and upper thresholds	T_low = 35%; T_high = 55%
Pump gain while ON	1.5 moisture points per step
Peak evapotranspiration	0.30 moisture points per step
Rainfall event	Day 2, 10:00–11:00; +15 points
Drainage above 60%	50% of excess per step
Fixed-schedule baseline	40 minutes at 06:00 and 18:00

4.3 Evaluation Metrics

The simulation records pump-on time, drainage loss, minimum and maximum moisture, time below a critical moisture level, and the percentage of time spent inside the target band. These metrics help compare control behavior under the same assumptions. The field protocol adds

measurements that cannot be established reliably from the simplified model, including sensor calibration error, relay actuation latency, telemetry delivery, water volume, and fault response.

4.4 Simulation Results

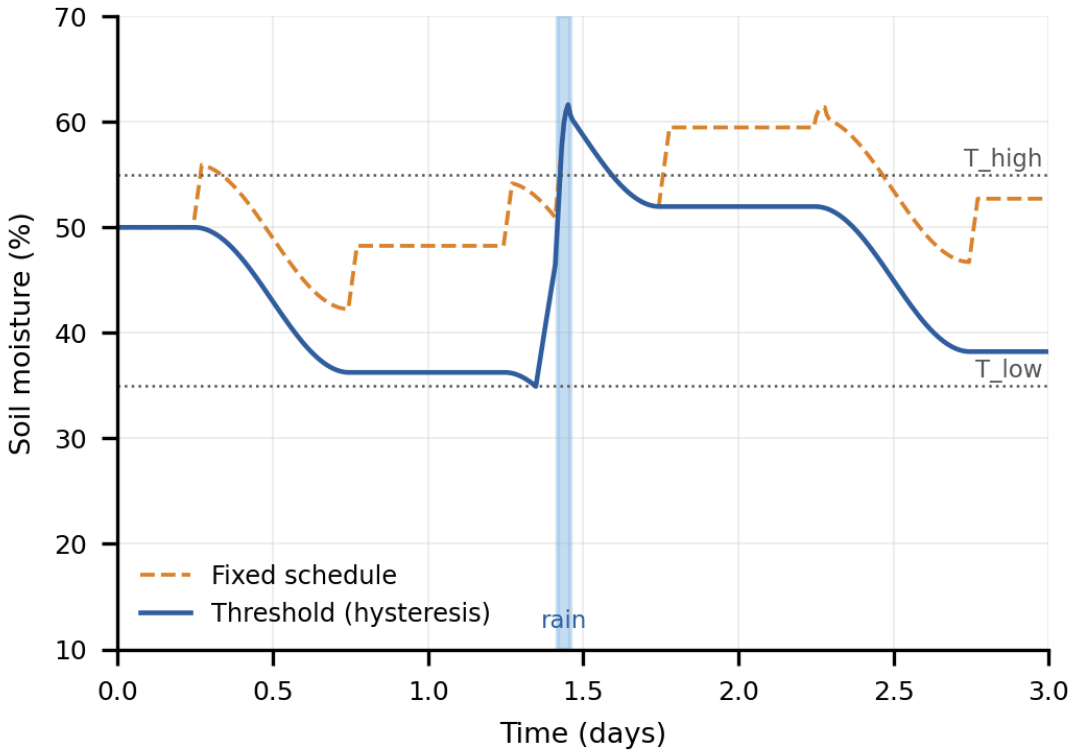


Figure 4.1: Simulated soil moisture under threshold control and a fixed schedule.

Table 4.4: Simulation results

Metric	Fixed Schedule	Threshold Control
Pump-on time	260 minutes	120 minutes
Drainage loss	10.0 moisture points	3.5 moisture points
Minimum moisture	42.3%	34.9%
Maximum moisture	61.6%	61.6%
Time below 30%	0 minutes	0 minutes
Time within 35–60% target band	97.2%	98.8%

In the assumed three-day scenario, the threshold controller uses 120 pump-on minutes, compared with 260 minutes for the fixed schedule. The relative reduction is approximately 53.8%, calculated as (260 − 120) / 260 × 100. This occurs because the controller avoids pumping when the modeled soil moisture is already adequate and responds to the rainfall event. The result demonstrates how feedback control can reduce unnecessary pump operation in this particular model.

The modeled drainage loss is 3.5 moisture points under threshold control, compared with 10.0 points under the fixed schedule. This is consistent with the controller avoiding some irrigation when moisture is already high. The time within the 35–60% target band is 98.8% for threshold control and 97.2% for the fixed schedule. Neither strategy falls below 30% during the modeled period. The threshold run reaches a minimum of 34.9%, near the lower threshold, because irrigation starts only after the moisture condition crosses the trigger.

These values should be interpreted cautiously. The pump gain, rainfall contribution, evapotranspiration, and drainage rule are assumed parameters. A different soil type, crop, sensor position, pump delivery rate, or weather pattern may produce different outcomes. Therefore, the simulation supports the functional behavior of the control logic but does not establish the water savings expected in a real field.

Table : 4.5 Planned Field Evaluation Protocol

Metric	**Measurement Method**	**Result**
Sensor calibration error	Compare sensor-derived moisture with oven-dry gravimetric samples	To be recorded
Actuation latency	Measure time from controller reading to relay state change	To be recorded
Telemetry delivery	Compare MQTT messages sent with messages received	To be recorded
Water used per day	Measure flow rate and pump-on duration; compare with timer baseline	To be recorded

Time in target band	Log moisture values between T_low and T_high	To be recorded
Fault handling	Test sensor unplugging, low tank, and Wi-Fi loss	To be recorded

Field testing should begin with a controlled bench test using a safe low-voltage pump supply. The moisture sensor should be calibrated for the intended soil, and the selected thresholds should be documented. A baseline timer schedule and the threshold controller should be compared over equivalent observation periods while recording moisture, pump runtime, and actual water volume. Repeated trials are recommended because rainfall, soil variation, and sensor placement can affect the results.

4.6 Discussion

The design keeps the core decision on the ESP32, so the irrigation response does not depend on continuous network availability. Hysteresis reduces rapid switching near a single set point, and the safety interlocks make important fault conditions end in the pump-off state. MQTT adds monitoring and remote configuration without requiring the dashboard to participate in every local decision.

The principal trade-off is simplicity versus prediction. Threshold control is easy to understand and calibrate, but it does not anticipate future weather or explicitly model crop-specific evapotranspiration. Sensor readings also represent only the sensor's local position and may not reflect moisture across an entire plot. The field evaluation must therefore confirm whether the selected thresholds and sensor placement are suitable for the target crop and soil.

4.7 Limitations

- No physical field measurements are reported in the current evaluation.
- The simulation uses assumed parameters and a simplified soil-water model.
- The moisture output is a relative index that depends on calibration and sensor placement.
- A low-cost temperature and humidity sensor may have limited accuracy.
- The design focuses on one irrigation zone and does not implement weather forecasting.
- Without a flow meter, actual water volume cannot be established from pump runtime alone.
- MQTT deployment requires authentication, TLS, and access controls for secure field use.

RESULTS

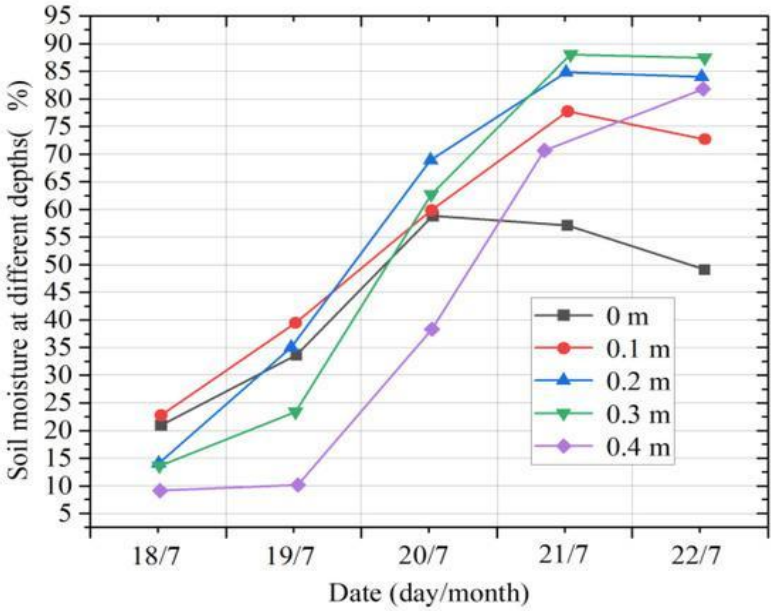


Figure 5.1: Soil Moisture Levels

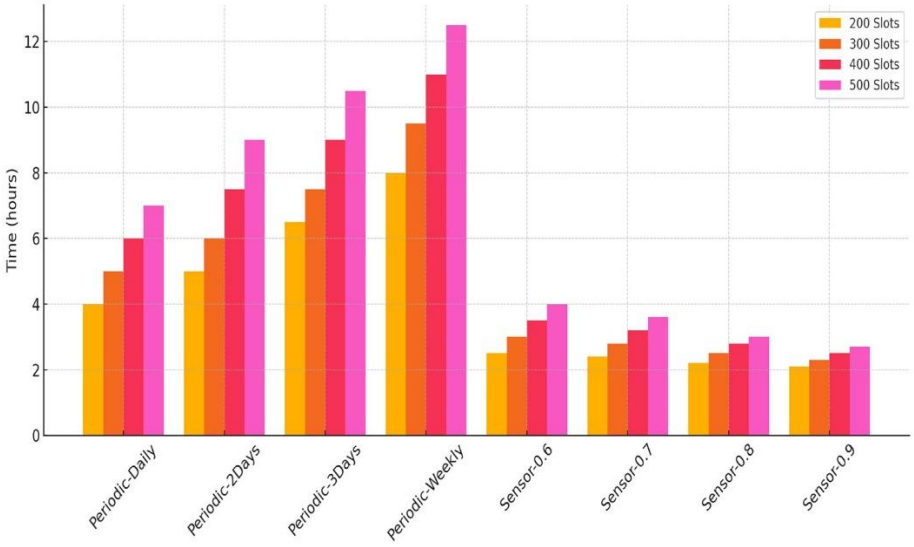


Figure 5.2: Five-Sensor Simulation

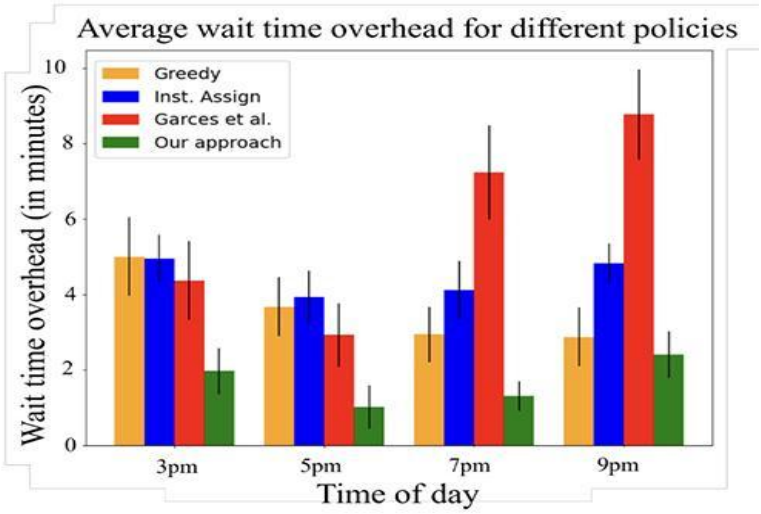


Figure 5.3: Water Usage Comparison

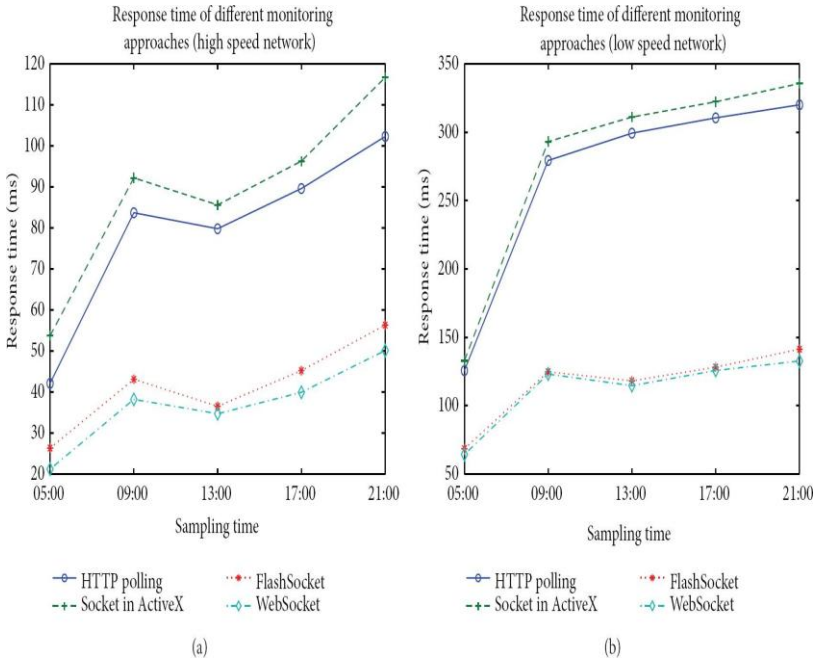


Figure 5.4: Sensor Polling Performance

CONCLUSION AND FUTURE WORK

This project presents a low-cost IoT-based smart irrigation system built around an ESP32, soil-moisture sensing, hysteresis control, safety interlocks, and MQTT telemetry. The controller measures the local soil condition and operates the pump using separate lower and upper thresholds. Invalid sensor readings, a low tank level, and excessive continuous pump runtime force the pump off, reducing the risk of unsafe operation.

In a three-day simulation using assumed parameters, threshold control required 120 pump-on minutes compared with 260 minutes for a fixed schedule. The threshold strategy also produced lower modeled drainage loss and a slightly higher proportion of time within the target moisture band. These findings describe the simulation only and must not be interpreted as verified field performance.

The project should therefore be viewed as a prototype design and simulation study, not as a completed field-validated agricultural product. Its practical performance will depend on calibration, sensor placement, pump flow, soil characteristics, crop requirements, and the reliability of the electrical installation. The next stage should prioritize a low-voltage physical prototype, a measured-water baseline comparison, and documented fault-injection tests. Only after those tests should the system's watersaving potential, operating cost, and suitability for a particular crop be quantified.

Future Work

- Build and test the physical prototype with calibrated sensors and a relay-driven pump.
- Use a flow meter to measure delivered water volume rather than estimating it from runtime.
- Collect repeated field measurements and compare threshold control with a fixed-schedule baseline.
- Extend the system to multiple zones using solenoid valves and independent moisture thresholds.
- Explore evapotranspiration-based scheduling using weather observations and crop coefficients.
- Add solar power and suitable energy management for remote installations.

REFERENCES

[1] C. Kamienski et al., “Smart water management platform: IoT-based precision irrigation for agriculture,” Sensors, vol. 19, no. 2, Art. no. 276, 2019, doi: 10.3390/s19020276.

[2] L. García, L. Parra, J. M. Jimenez, J. Lloret, and P. Lorenz, “IoT-based smart irrigation systems: An overview on the recent trends on sensors and IoT systems for irrigation in precision agriculture,” Sensors, vol. 20, no. 4, Art. no. 1042, 2020.

[3] L. Hamami and B. Nassereddine, “Application of wireless sensor networks in the field of irrigation: A review,” Computers and Electronics in Agriculture, vol. 179, Art. no. 105782, 2020.

[4] D. Thote, V. Lanjewar, V. Sharma, P. Agrawal, and V. K. Soni, “IoT and machine learning-based smart soil irrigation farming systems,” in Proc. IEEE SCEECS, 2024.

[5] Espressif Systems, “ESP32 Series Datasheet,” Espressif Systems. [Online]. Available: https://www.espressif.com

[6] A. Banks and R. Gupta, Eds., “MQTT Version 3.1.1,” OASIS Standard, Oct. 2014.

[7] S. I. Abba, J. W. Namkusong, J.-A. Lee, and M. L. Crespo, “Design and performance evaluation of a low-cost autonomous sensor interface for a smart IoT-based irrigation monitoring and control system,” Sensors, vol. 19, no. 17, Art. no. 3643, 2019.

[8] A. D. Boursianis et al., “Smart irrigation system for precision agriculture—The ARETHOU5A IoT platform,” IEEE Sensors Journal, vol. 21, pp. 17539–17547, 2021.

[9] R. Liao et al., “Development of smart irrigation systems based on real-time soil moisture data in a greenhouse: Proof of concept,” Agricultural Water Management, vol. 245, Art. no. 106632, 2021.

[10] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith, Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements, FAO Irrigation and Drainage Paper 56. Rome, Italy: FAO, 1998.

[11] R. A. Light, “Mosquitto: Server and client implementation of the MQTT protocol,” Journal of Open Source Software, vol. 2, no. 13, p. 265, 2017.

[12] K. J. Åström and R. M. Murray, Feedback Systems: An Introduction for Scientists and Engineers. Princeton, NJ, USA: Princeton University Press, 2008.

.